

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 4**



SORTING

Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin

NIM. 2010817110011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 4

Laporan Praktikum Algoritma & Struktur Data Modul 4: Sorting ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin
NIM : 2010817110011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL.....	iv
SOAL.....	1
A. Source Code.....	2
B. Pembahasan.....	10
TAUTAN GIT.....	22

DAFTAR GAMBAR

DAFTAR TABEL

<i>Table 1. Source Code single_linked_list.cpp.....</i>	<i>10</i>
<i>Table 2. Table waktu eksekusi.....</i>	<i>18</i>
<i>Table 3. Table algoritma tercepat.....</i>	<i>19</i>
<i>Table 4. Table algoritma sensitif bentuk data.....</i>	<i>20</i>
<i>Table 5. Table algoritma sensitif bentuk data waktu.....</i>	<i>20</i>
<i>Table 6. Table hubungan teori dan hasil.....</i>	<i>21</i>

SOAL

Sorting

Tujuan Praktikum

1. Mahasiswa mampu mengimplementasikan berbagai algoritma sorting (Comparison & Non-comparison).
2. Mahasiswa mampu melakukan analisis performa menggunakan waktu eksekusi (ms), jumlah perbandingan (comparisons), dan jumlah pertukaran (swaps).
3. Mahasiswa memahami pengaruh distribusi data terhadap efisiensi algoritma.

Ketentuan Tugas

- Implementasi dikerjakan secara modular menggunakan template yang disediakan.
- Dilarang keras menggunakan library `std::sort`. Semua algoritma harus ditulis manual.
- Dilarang menggunakan AI. Ini kalian harus analisis sendiri. Baca dulu dari sumber-sumber buku, materi, internet yang diberikan
- Sortingnya dengan urutan menaik atau ascending
- Data yang akan diurutkan berupa `vector<int>`

Algoritma Sorting

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort
- Radix Sort

Instruksi Pengujian

Lakukan pengujian terhadap setiap algoritma menggunakan beberapa kondisi data berikut:

1. Data acak (random).
2. Data sudah (atau hampir) terurut menaik.
3. Data terurut menurun.

4. Data dengan banyak elemen duplikat.

Hal yang Perlu Dianalisis

Untuk setiap algoritma, lakukan analisis terhadap aspek berikut:

1. Kompleksitas waktu:
 - Best Case
 - Average Case
 - Worst Case
2. Karakteristik algoritma:
 - Apakah algoritma bersifat stable atau tidak.
 - Apakah algoritma termasuk in-place atau membutuhkan memori tambahan.
 - Mekanisme utama algoritma dalam melakukan pengurutan.
3. Analisis operasi:
 - Jumlah comparison.
 - Jumlah swap.
 - Jumlah shift (khusus algoritma tertentu seperti Insertion Sort).
 - Jumlah recursion call.
 - Jumlah array accessed.
 - Pengaruh bentuk data terhadap jumlah operasi.
4. Analisis performa:
 - Perbandingan waktu eksekusi antar algoritma.
 - Algoritma mana yang paling cepat untuk dataset kecil dan besar.
 - Pengaruh distribusi data terhadap performa algoritma.
 - Hubungan antara teori kompleksitas dan hasil eksperimen yang diperoleh.

A. Source Code

sorting_algorithms.cpp

1	#include "sorting_algorithms.h"
---	---------------------------------

```

2  #include <algorithm>
3  #include <chrono>
4
5  using Clock = std::chrono::high_resolution_clock;
6
7  void bubble_sort(std::vector<int> &data, Metrics &m) {
8      int n = data.size();
9      auto start = Clock::now();
10
11     // Done: implementasi Bubble Sort
12     for (int i = 0; i < n - 1; i++) {
13         // Loop kedua untuk membandingkan elemen
14         for (int j = 0; j < n - i - 1; j++) {
15             m.comparisons++;
16             // Jika elemen lebih besar dari elemen berikutnya,
17             tukar
18             if (data[j] > data[j + 1]) {
19                 std::swap(data[j], data[j + 1]);
20                 m.swaps++;
21             }
22         }
23     }
24
25     m.time_ms =
26                 std::chrono::duration<double,
27 std::milli>(Clock::now() - start).count();
28 }
29
30 void selection_sort(std::vector<int> &data, Metrics &m)
31 {
32     int n = data.size();
33     auto start = Clock::now();
34
35     // Done: implementasi Selection Sort
36     for (int i = 0; i < n - 1; i++) {
37         // Asumsi elemen pertama adalah elemen terkecil
38         int min_idx = i;
39         // Loop kedua untuk mencari elemen terkecil di
40         array
41         for (int j = i + 1; j < n; j++) {
42             m.comparisons++;
43             // Jika elemen lebih kecil dari elemen terkecil,
44             update elemen terkecil
45             if (data[j] < data[min_idx]) {

```



```

46         min_idx = j;
47     }
48 }
49 // Tukar elemen terkecil dengan elemen pertama
50 std::swap(data[i], data[min_idx]);
51 m.swaps++;
52 }
53 // Gunakan m.comparisons++ dan m.swaps++
54
55 m.time_ms =
56             std::chrono::duration<double,
57 std::milli>(Clock::now() - start).count());
58 }
59
60 void insertion_sort(std::vector<int> &data, Metrics &m)
61 {
62     int n = data.size();
63     auto start = Clock::now();
64
65     // Done: implementasi Insertion Sort
66     for (int i = 1; i < n; i++) {
67         int key = data[i];
68         int j = i - 1;
69
70         while (j >= 0 && data[j] > key) {
71             m.comparisons++;
72             // Geser elemen ke kanan
73             data[j + 1] = data[j];
74             m.shifts++;
75             j--;
76         }
77         // Masukkan key ke posisi yang tepat
78         data[j + 1] = key;
79         m.shifts++;
80     }
81     // Gunakan m.comparisons++ dan m.shifts++
82
83     m.time_ms =
84             std::chrono::duration<double,
85 std::milli>(Clock::now() - start).count());
86 }
87
88 void merge(std::vector<int> &data, int left, int mid, int
89 right, Metrics &m) {

```

```

90 // Mencari ukuran sub array left dan right
91 int n1 = mid - left + 1;
92 int n2 = right - mid;
93
94 // Membuat sub array
95 std::vector<int> L(n1), R(n2);
96 // Memasukkan data ke sub array
97 for (int i = 0; i < n1; i++)
98     L[i] = data[left + i];
99 for (int j = 0; j < n2; j++)
100     R[j] = data[mid + 1 + j];
101 // Variable helper untuk track index sub array dan main
102 array
103     int i = 0, j = 0, k = left;
104     // Membandingkan elemen sub array L dan R, lalu
105     memasukkan ke main array
106     while (i < n1 && j < n2) {
107         m.comparisons++;
108         // Jika elemen sub array L lebih kecil dari elemen
109         sub array R
110         // Maka masukkan elemen sub array L ke main array
111         // Jika tidak, masukkan elemen sub array R ke main
112         array
113         if (L[i] <= R[j]) {
114             // Memasukkan elemen sub array L ke main array
115             data[k] = L[i];
116             i++;
117         } else {
118             // Memasukkan elemen sub array R ke main array
119             data[k] = R[j];
120             j++;
121         }
122         k++;
123     }
124     // Memasukkan elemen sub array L yang tersisa
125     while (i < n1) {
126         data[k] = L[i];
127         i++;
128         k++;
129     }
130     // Memasukkan elemen sub array R yang tersisa
131     while (j < n2) {
132         data[k] = R[j];
133         j++;

```

```

134     k++;
135 }
136 }
137
138 void merge_sort_helper(std::vector<int> &data, int left,
139 int right,
140                     Metrics &m) {
141     m.recursive_calls++;
142
143     // Basis kasus: Jika sub array kosong atau memiliki 1
144     elemen, maka sudah
145     // terurut
146     if (data.empty())
147         return;
148     if (left >= right)
149         return;
150     // Membagi array menjadi 2 bagian
151     int mid = left + (right - left) / 2;
152     // Memanggil recursive function untuk sub array kiri
153     merge_sort_helper(data, left, mid, m);
154     // Memanggil recursive function untuk sub array kanan
155     merge_sort_helper(data, mid + 1, right, m);
156     // Menggabungkan sub array yang sudah terurut
157     merge(data, left, mid, right, m);
158 }
159
160 void merge_sort(std::vector<int> &data, Metrics &m) {
161     auto start = Clock::now();
162
163     // Done: panggil helper rekursif di sini
164     merge_sort_helper(data, 0, data.size() - 1, m);
165     // Gunakan m.comparisons++ dan m.recursive_calls++
166
167     m.time_ms =
168                 std::chrono::duration<double,
169 std::milli>(Clock::now() - start).count();
170 }
171
172 int quick(std::vector<int> &data, int low, int high,
173 Metrics &m) {
174     int pivot = data[high];
175     int i = low - 1;
176
177     // loop i = -1 j = 0

```

```

178 // data = [5, 2, 8, 1, 9, 3, 7]
179 // pivot = 7
180 // it 1 = no swap
181 // it 2 = no swap i = 1 j = 1
182 // it 3 = j = 2 i = 1 Skip
183 // it 4 = j= 3 i = 2 swap data[2], data[3] = [5, 2, 1,
184 8, 9, 3, 7]
185 // it 5 = j = 4 i = 2 skip
186 // it 6 = j = 5 i = 3 swap data[3], data[5] = [5, 2, 1,
187 3, 9, 8, 7]
188 for (int j = low; j < high; j++) {
189     m.comparisons++;
190     if (data[j] <= pivot) {
191         i++;
192         if (i != j) {
193             std::swap(data[i], data[j]);
194             m.swaps++;
195         }
196     }
197 }
198 // swap data[4] dan data[6] = [5, 2, 1, 3, 7, 8, 9]
199 std::swap(data[i + 1], data[high]);
200 m.swaps++;
201 // pi 4
202 return i + 1;
203 }
204
205 void quick_sort_helper(std::vector<int> &data, int low,
206 int high, Metrics &m) {
207     m.recursive_calls++;
208     if (low < high) {
209         // pi 4
210         int pi = quick(data, low, high, m);
211         // recursive calls di sini
212         // pivot baru jadi high nya left dan low nya right
213         // left = [5, 2, 1, 3] pi-1 = 3
214         // it 1 = no swap j = 0 i = -1
215         // it 2 = j = 1 i = 0 swap data[0], data[1] = [2, 5,
216 1, 3]
217         // it 3 = j = 2 i = 1 swap data[1], data[2] = [2, 1,
218 5, 3]
219         // swap data[2], data[3] = [2, 1, 3, 5]
220         // pi 2
221         // Recursive calls di sini lagi

```

```

222     quick_sort_helper(data, low, pi - 1, m);
223     // right = [8, 9, 7] high = 6 pi+1 = 5
224     quick_sort_helper(data, pi + 1, high, m);
225 }
226 }
227
228 void quick_sort(std::vector<int> &data, Metrics &m) {
229     auto start = Clock::now();
230
231     // Done: panggil helper rekursif di sini
232     quick_sort_helper(data, 0, data.size() - 1, m);
233     // Gunakan m.comparisons++, m.swaps++, dan
234     m.recursive_calls++
235
236     m.time_ms =
237         std::chrono::duration<double,
238         std::milli>(Clock::now() - start).count();
239 }
240
241 // TODO: tambahkan helper per-digit sort di sini jika
242 perlu
243 void counting_sort_by_digit(std::vector<int> &data, int
244 exp, Metrics &m) {
245     int n = data.size();
246     std::vector<int> output(n);
247     int count[10] = {0};
248     // Awal data = [170, 45, 75, 90, 2, 802, 24, 66]
249     //      Index  0   1   2   3   4   5   6   7
250
251     // exp = 10
252     // [170, 90, 2, 802, 24, 45, 75, 66]
253     // Hitung frekuensi digit
254     for (int i = 0; i < n; i++) {
255         int digit = (data[i] / exp) % 10;
256         count[digit]++;
257         m.array_accesses++;
258     }
259     // count = [2, 0, 1, 0, 1, 0, 1, 2, 0, 1, 0]
260     //      [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
261
262     // Prefix sum untuk posisi
263     for (int i = 1; i < 10; i++) {
264         count[i] += count[i - 1];
265         m.array_accesses++;

```

```

266 }
267 // count = [2, 2, 3, 3, 4, 4, 5, 7, 7, 8, 8]
268 // index = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
269
270 // Build output vector (traverse dari belakang agar
271 stabil)
272 for (int i = n - 1; i >= 0; i--) {
273     int digit = (data[i] / exp) % 10;
274     output[count[digit] - 1] = data[i];
275     count[digit]--;
276     m.array_accesses++;
277 }
278 // count = [0, 2, 2, 3, 3, 4, 4, 5, 7, 7, 8]
279 // index = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
280
281 // output = [2, 802, 24, 45, 66, 170, 75, 90]
282 // Index    0    1    2    3    4    5    6    7
283
284 // Copy kembali ke data
285 for (int i = 0; i < n; i++) {
286     data[i] = output[i];
287     m.array_accesses++;
288 }
289 }
290
291 // exp 100
292 // output = [2, 802, 24, 45, 66, 170, 75, 90]
293 // freq digi : for (int i = 0; i < n); int digit =
294 (data[i] / exp) % 10;
295 // count[digit]++;
296 // count = [6, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0]
297 // index   [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
298 // pref sum count : for (int i = 1; i < 10; i++) count[i]
299 += count[i - 1];
300 // count = [6, 7, 7, 7, 7, 7, 7, 7, 8, 8, 8]
301 // index   [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
302 // Build output : for (int i = n - 1; i >= 0; i--)
303 // int digit = (data[i] / exp) % 10; output[count[digit]
304 - 1] = data[i];
305 // count[digit]--;
306 // count = [0, 6, 7, 7, 7, 7, 7, 7, 7, 8, 8]
307 // index   [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
308 // output = [2, 24, 45, 66, 75, 90, 170, 802]
309 // Index   [0, 1, 2, 3, 4, 5, 6, 7]

```

```

310 // Sorted
311 void radix_sort(std::vector<int> &data, Metrics &m) {
312     if (data.empty())
313         return;
314     auto start = Clock::now();
315
316     // Done: implementasi Radix Sort
317     // mencari max element
318     int max_val = *std::max_element(data.begin(),
319 data.end());
320     for (int exp = 1; max_val / exp > 0; exp *= 10) {
321         counting_sort_by_digit(data, exp, m);
322     }
323     // Gunakan m.array_accesses++
324
325     m.time_ms =
326         std::chrono::duration<double,
327 std::milli>(Clock::now() - start).count();
328 }

```

Table 1. Source Code single_linked_list.cpp

B. Pembahasan

Bubble Sort

Kompleksitas waktu

1. **Best Case** : Kasus terbaik untuk bubble sort adalah pada bentuk data yang sudah urut dengan jumlah swap 0 dan waktu 0.003ms.
2. **Average Case** : Kasus rata – rata dari algoritma ini berada pada bentuk data yang banyak duplikat. Dengan jumlah swap 2315 dan waktu 0.02ms.
3. **Worst Case** : Kasus terburuk dari algoritma ini adalah pada bentuk data reverse sorted dengan jumlah swap 4950 dan waktu 0.029ms

Untuk Big-O notation nya adalah $O(n^2)$. Karena terdapat nested loop sebanyak n pada algoritmanya pada baris [12,14], dan tidak ada perubahan pada jumlah comparisons dari semua jenis bentuk data yaitu 4950. membuktikan bahwa tidak ada perbedaan notasi walaupun bentuk data berubah.

Karakteristik algoritma

Apakah algoritma bersifat stable karena element yang sama posisi nya tidak akan dilakukan pertukaran, dan posisi nya tetap relatif pada sorting, hal ini karena pada comparasi pada baris [18] algoritma menggunakan > sehingga jika nilai sama maka tidak akan di swap. Algoritma ini inplace karena dia tidak memerlukan array / vector / container bantuan dengan besar sebanyak n, dan langsu melakukan sorting pada vector / data aslinya.

Algoritma ini bekerja dengan membandingkan semua data satu persatu data yang ada dengan data disebelahnya, jika nilai pada index saat ini lebih kecil dari data setelah nya swap mereka terus menerus sampai algoritma selesai membandingkan semua nilai setiap index pada data

Analisis operasi

Comparison : 4950 untuk semua bentuk data

Swap : Random sebanyak 2564, Sorted sebanyak 0, Reverse Sorted sebanyak 4950, Nearly Sorted sebanyak 296, Many Duplicates sebanyak 2315.

Pengaruh bentuk data terhadap jumlah operasi :

Data terurut menaik : ini menjadi best case karena algoritma tidak perlu lagi melakukan swap karea data sudah terurut dan ini disebabkan oleh comparasi yang ada pada baris [18].

Data terurut menurun : menjadi worst case karena data akan diswap sama banyaknya dengan comparison karena hasil dari comparison nya akan selalu true sehingga swap selalu dilakukan.

Data acak : terlihat hasil data acak memiliki swap yang di atas rata – rata hal ini karena jika data sama maka swap tidak akan dilakukan hal ini membuat swap secara natural lebih dikit dari swap data banyak duplikat.

Data banyak duplikat : ini mejadi kasus rata – rata karena ada beberapa swap yang tidak dilakukan sebab comaprasi yang memiliki hasil false sehingga ada beberapa ini menjadi lebih dikit dari random.

Selection Sort

Kompleksitas waktu

1. **Best Case** : Sorted menjadi best case dengan waktu 0.003296ms.
2. **Average Case** : Reverse Sorted average case dengan waktu 0.004057ms.
3. **Worst Case** : Random menjadi worst case dengan waktu 0.006433ms.

Untuk algoritma ini sebenarnya perbedaan hanya terdapat pada waktu comparasi saja, Big-O notation dari algoritma ini adalah $O(n^2)$ karena adanya nested loop sebanyak n pada baris [36, 41], dan tidak ada perubahan notasi walaupun bentuk data berbeda karena comparasi dan swap memiliki nilai yang sama pada semua bentuk data.

Karakteristik algoritma

Algoritma ini bersifat tidak stable karena dia akan menukar index data jika nilai nya sama jadi posisi relatif dari nilai yang sama tidak bisa ditentukan. Algoritma termasuk in-place karena tidak memerlukan array / vector (container n) bantuan dia hanya memerlukan 1 variable bantuan pada baris [33].

Pertama algoritma ini akan mengasumsi bahwa index pertama adalah nilai paling kecil. Kemudian akan dilakukan perulangan i untuk mengisi setiap index dari data, yang didalam nya ada perulangan j lagi untuk mencari nilai terkecil. Pada perulangan mencari nilai terkecil ini terdapat comparasi dimana jika nilai index saat ini lebih kecil maka nilai terkecil di update untuk menyimpan nilai index yang baru. Dan jika sudah semua index di cek swap nilai index terkecil dengan index i (index perulangan i saat ini). Dan ini dilakukan terus sampai semua index pada perulangan i sudah selesai.

Analisis operasi

Comparison : 4950 untuk semua bentuk data

Swap : 99 untuk semua bentuk data.

Pengaruh bentuk data terhadap jumlah operasi pada algoritma ini tidak terlalu berdampak karena bagaimana pun bentuk data nya algoritma ini akan selalu melakukan swap yang sama yaitu sebanyak $n-1$, yang menjadi pembeda antara bentuk data disini hanya pada waktunya, dimana Random 0.006ms, Sorted 0.003ms, Reverse Sorted 0.004ms, Nearly Sorted 0.003ms, Many Duplicates 0.004ms. Hal ini terjadi karena waktu comparison, jika makin random maka makin lama waktu comparison nya.

Insertion Sort

Kompleksitas waktu

1. **Best Case** : $O(n)$. Sorted menjadi best case karena comparisons nya 0 dan shifts nya 99.

2. **Average Case** : $O(n^2)$. Random menjadi average case karena comparisons nya 2564 dan shifts nya 2663.
3. **Worst Case** : $O(n^2)$. Reverse Sorted menjadi worst case karena comparisons nya 4950 dan shifts nya 5049.

Kenapa case nya ini sangat berbeda pada bentuk data nya?, ini karena algoritma ini menggunakan perulangan berkondisi yaitu while pada baris [70-80]. ini membuat jika key lebih besar dari data j maka shifts tidak perlu dilakukan, dan semakin kanan kamu perlu melakukan while shifts ini makan makin banyak comparison dan shifts nya. Karena ini Big-O Notation nya berbeda pada Best case $O(n)$, karena kita tidak perlu melakukan loop while, dan pada average dan worst $O(n^2)$ karena loop while dijalankan.

Karakteristik algoritma

Algoritma ini bersifat stable karena comparison nya menggunakan $>$ dimana ini membuat algoritma tidak akan melakukan shift jika data dan key nya sama dia tidak akan melakukan shift. Algoritma ini termasuk in-place karena algoritma ini tidak membuat container pembantu sebesar n dan sorting dilakukan pada container data asli..

Pertama algoritma ini akan melakukan perulangan sebanyak n , yang membuat perulangan ini menarik adalah dia mulai dari 1 tidak 0, karena key mulai dari index 1. perulangan ini digunakan sebagai bagian utama algoritma. Didalam perulangan itu algoritma membuat variable bantu bernama key dan j. Key digunakan untuk menyimpan data pada index 1, dan j digunakan untuk menyimpan index dari $I - 1$ dari perulangan pertama. Kemudian ada perulangan while dengan kondisi $j \geq 0$ dan data index $j > \text{key}$. Selama kondisi benar maka kita majukan data index nya dengan membuat data index $j+1 = \text{data } j$ dan mengurangi nilai $j - 1$ dengan begitu index belakang j akan maju 1 index selama kondisi benar. Jika kondisi sudah tidak terpenuhi maka lanjut untuk masukan data index $j + 1$ setelah j menjadi -1 jadi kita kembalikan menjadi 0 dan kita isi dengan nilai key.

Analisis operasi

Comparison : Random sebanyak 2564, Sorted sebanyak 0, Reverse Sorted sebanyak 4950, Nearly Sorted sebanyak 296, Many Duplicates sebanyak 2315.

Shifts : Random sebanyak 2663, Sorted sebanyak 99, Reverse Sorted sebanyak 5049, Nearly Sorted sebanyak 395, Many Duplicates sebanyak 2414.

Pengaruh bentuk data terhadap jumlah operasi:

Data terurut menaik : Sorted memiliki comparison dan shifts paling rendah karena while loop langsung false. Loop dan shifts nya nggak jalan.

Data terurut menurun : Reverse Sorted menjadi yang worst karena key harus geser lewati semua elemen di kiri.

Data acak : Random ini menjadi average karena key kadang tidak perlu geser sepenuhnya hanya sampe tempatnya.

Data banyak duplikat : Banyak duplikat ini ditengah – tengah karena kalau nilai sama, kondisi $> \text{false}$. Jadi elemen nggak perlu geser melewati elemen yang nilainya sama.

Kalau datanya udah rapi (sorted), Insertion Sort ini cepat. Comparisons sama shifts-nya cuma sekitar 99. Tapi kalau datanya terbalik, dia langsung lemot. Tiap angka baru harus diselipin ke paling depan, jadi key dan datanya shifts terus. Untuk data acak, dia di tengah-tengah aja. Yang menarik, kalau data banyak angka yang sama (duplikat), dia sedikit lebih cepat karena tidak perlu tuker posisi kalau nilainya sama.

Merge Sort

Kompleksitas waktu

1. **Best Case, Average Case, Worst Case** : Sama karena Recursive Calls nya sama pada semua bentuk data.

Karena bagaimana pun datanya, algoritma ini tetep bagi 2, bagi 2, bagi 2... terus merge. Nggak ada jalan pintas. Jadi jumlah operasi nya relatif konstan. Tapi kalau dilihat dari jumlah comparisons pas merge, ada perbedaan kecil. Meski begitu, perbedaannya cuma ratusan, nggak mengubah notasi Big-O yang tetep $O(N \log N)$.

Karakteristik algoritma

Algoritma bersifat stable karena merge, adanya comparasi if ($L[i] \leq R[j]$). Tanda $\leq$ ini karena jika elemen kiri dan kanan nilainya sama, yang dari kiri (L) yang diambil duluan. Jadi posisi relatif elemen bernilai sama nggak berubah di datanya. Algoritma ini tidak in-place karena algoritma ini perlu vector bantuan (L dan R). dan juga recursive calls nya sebanyak n juga. Space complexity-nya $O(N)$ karena ini tidak nested recursive calls dan vector nya. karena bergantung sebanyak n walaupun dibagi-bagi vectornya

Algoritma ini bekerja dengan cara memanggil fungsi recursive yang pada kasus ini adalah helper yang memiliki parameter data, left, right. Dimana pemanggilan pertama left nya adalah 0 dan right nya data size -1. ini adalah panggilan awal dimana data belum dibagi. Panggilan ini kita ibaratkan menjadi stack panggilan fungsi. Sebelum itu fungsi helper disini

memiliki cek apakah data empty dan cek $\text{left} \geq \text{right}$ dimana size nya disini 1 dia akan return dan mematikan jalan fungsi, dan fungsi ini juga membuat var mid yang berfungsi sebagai pembagi data ditengah nya. Kemudian fungsi ini juga memanggil dirinya sendiri 2x dengan parameter yang berbeda dimana yang kiri `merge_sort_helper(data, left, mid, m)`; right nya menjadi mid dan yang kanan `merge_sort_helper(data, mid + 1, right, m)`; dimana left nya menjadi mid + 1. artinya fungsi ini akan terus membagi vector data sampai vector datanya hanya 1. kemudian fungsi ini juga memanggil merge yang akan melakukan merge vector data.

Jadi alur nya adalah helper utama masuk ke stack, dia akan memanggil helper 1 kiri dlu dan masuk stack, dan helper 2 kiri dlu, nah jika helper 2 kiri ini size nya 1 dia akan return dan menutup stack ini, kemudian pada helper 1 kiri akan memanggil helper 2 kanan, dan jika size 1 juga kembali, baru dia akan merge dengan parameter helper 1 kiri dan seterusnya sampai merge pada helper utama.

Untuk merge? Algoritma merge biasa.

Analisis operasi

Comparison : Random sebanyak 535, Sorted sebanyak 356, Reverse Sorted sebanyak 316, Nearly Sorted sebanyak 453, Many Duplicates sebanyak 536.

Rec.Calls : Random sebanyak 199, Sorted sebanyak 199, Reverse Sorted sebanyak 199, Nearly Sorted sebanyak 199, Many Duplicates sebanyak 199.

Pengaruh bentuk data terhadap jumlah operasi.

Untuk Merge Sort, bentuk data tidak terlalu berpengaruh ke jumlah operasi secara signifikan. Recursive calls-nya tetap 199 dengan $N=100$. Namun comparison nya yang sedikit berbeda hal ini terjadi karena jika datanya udah sorted atau reverse, saat merge salah satu subvector cepat habis, jadi comparisons nya lebih dikit. Jika datanya acak dan elemennya saling berselipan, comparisons nya jadi maksimal karena loop while jalan terus.

Quick Sort

Kompleksitas waktu

1. **Best Case** : $O(N \log N)$. Random dan Many duplicate menjadi base case, karena pivot selalu membagi vector jadi 2 bagian yang seimbang (sekitar $n/2$).

2. **Average Case** : $O(N \log N)$. Random menjadi case yang average karena partisinya cenderung seimbang.
3. **Worst Case** : $O(N^2)$. Sorted dan Reverse Sorted menjadi case paling buruk karena pivot selalu jadi elemen terbesar/terkecil. Partisi nggak seimbang, dan tidak bisa di partisi, jadi recursive depth-nya jadi N .

Karakteristik algoritma

Algoritma ini bersifat tidak stable. Karena saat partition quick, elemen bisa loncat posisi lewat nilai i dan j , jadi urutan relatif elemen yang nilainya sama bisa berubah.

Algoritma ini termasuk in-place (sorting dilakukan di vector asli), tapi butuh stack memori untuk recursive calls .

Pertama ada fungsi quick disini sebagai fungsi sorting nya, pertama fungsi ini memiliki parameter data, low, dan high. Pada fungsi ini kita perlu var pivot dengan value = high dalam contoh ini 6, dan $I = \text{low} - 1$ ($\text{low} = 0$) / $I = -1$. kemudian ada perulangan utama dari fungsi ini dengan kondisi $\text{int } j = \text{low}; j < \text{high}; j++$, didalam nya ada comparison $\text{data}[j] \leq \text{pivot}$ dimana jika true nilai I akan di increment dan akan dilakukan $\text{swap}(\text{data}[i], \text{data}[j])$, increment I hanya akan jalan apabila $j \leq \text{pivot}$ ini menyebabkan kadang I tertinggal oleh j yang tetap terus di increment ini membuat swap tidak swap dengan dirinya sendiri. Masih di fungsi ini setelah perulangan selesai akan ada swap ($\text{data}[i + 1], \text{data}[\text{high}]$), I disini nilai baru setelah increment dari loop, dan kemudian fungsi ini akan return $i + 1$ yang nantinya digunakan sebagai pivot baru.

Sekarang ke fungsi helper tempat dimana ada nya recursive calls untuk membagi panjang vector sehingga fungsi quick disini tidak perlu melakukan banyak iterasi di loop nya karena panjang vector yang di pangkas dengan mid nya sesuai dengan pivot. Pertama didalam fungsi ini ada cek jika $\text{low} < \text{high}$, dan didalam nya ada var pi yang menampung hasil dari quick yang saya jelaskan diatas, dan ada 2 recursive calls yaitu left dan right, yang membedakan dari merge disini mid nya akan di isi oleh pi yang nanti nya jadi pivot baru di quick.

Analisis operasi

Comparison: Random sebanyak 676, Sorted sebanyak 4950, Reverse Sorted sebanyak 4950, Nearly Sorted sebanyak 2638, Many Duplicates sebanyak 832, comp ini terjadi di setiap perbandingan elemen dengan pivot di dalam loop partition quick baris [190].

Swap: Random sebanyak 396, Sorted sebanyak 5049, Reverse Sorted sebanyak 2549, Nearly Sorted sebanyak 2607, Many Duplicates sebanyak 707, terjadi saat elemen dipindahkan ke posisi i baris [193] dan saat pivot dipasang di posisi akhir baris [199].

Recursion call: Random sebanyak 131, Sorted sebanyak 199, Reverse Sorted sebanyak 199, Nearly Sorted sebanyak 191, Many Duplicates sebanyak 181, terjadi setiap masuk ke quick_sort_helper baris [222, 224], termasuk base case.

Pengaruh bentuk data terhadap jumlah operasi.

Dari hasil matriks nya ini quick sort cenderung susah di tipe data yang sudah terurut karena pivot akan selalu mengambil right yang dimana jika datanya terurut maka biasanya pivot menjadi yang paling besar atau kecil ini membuat partisi nya tidak imbang atau bahkan hanya bisa menghasilkan 1 partisi.

Radix Sort

Kompleksitas waktu

1. **Best Case, Average Case, Worst Case:** Tidak ada perbedaan yang didasarkan tipe data, hal ini karena sort dilakukan seberapa banyak tergantung dengan digit dari max element pada baris [254, 255], dan loop didalam sort akan tetap dilakukan sebanyak n kali tanpa kondisi apapun. Dengan begitu kompleksitas waktu nya $O(n \times d)$, d adalah digit max element, dan Big-O notation nya menjadi $O(n)$ karena d adalah static tidak tergantung n .

Karakteristik algoritma

Algoritma ini bersifat stable. Karena counting sort yang digunakan membangun output dari belakang (traverse mundur), sehingga elemen yang muncul lebih dulu tetap didepan karena yang belakang dimasukan terlebih dahulu.

Algoritma ini bukan in-place. Dia butuh container bantuan vector output dan array count[10], sehingga kompleksitas ruang nya $O(n + k)$ dengan $k=10$, Big-O notation nya menjadi $O(n)$

Algoritma ini sangat berbeda dari algoritma sebelumnya karena algoritma ini tidak menggunakan comparison terhadap dua data sama sekali, tetapi dia langsung mengurutkan data berdasarkan nilai satuan, puluhan, ratusan, dan seterusnya, data. Algoritma ini perlu tahu element terbesar dari data yang ingin di sort, data ini akan menjadi patokan seberapa banyak algoritma ini akan dijalankan. Algoritma ini memiliki parameter data dan exp, exp ini adalah 10 dan iterasi *10, jadi jika max value nya ratusan maka akan jalan 3x.

Algoritma ini memerlukan 2 container bantuan, 1 container array count dengan size 10, dan 1 vector dengan size n , algoritma ini akan melakukan 4 loop dengan tujuan yang berbeda. Loop pertama untuk mencari frequensi digit (digit nilai di data) yang akan dimap ke count, loop ini berjalan dengan kondisi $i < n$ dengan iterasi $i++$, pertama pada loop ini perlu mencari

digit data dengan cara $(data[i] / exp) \% 10$, kemudian nilai $count[digit]$ di tambah 1. Loop kedua untuk menjumlahkan nilai yang ada pada count, loop ini berjalan dengan $i < 10$; $i++$, didalam nya kita akan memasukan nilai $count[i]$ menjadi $count[i]+count[i - 1]$. Loop ketiga untuk melakukan asign data ke vector output, loop ini berjalan dengan $i = n - 1$; $i \geq 0$; $i--$, ya loop ini mulai dari belakang agar dia lebih stabil karena jika ada element yang sama posisi yang didepan akan relatif tetap didepan antara element yang sama, loop ini tetap memerlukan digit dari data $(data[i] / exp) \% 10$, kemudian lakukan asign $output[count[digit] - 1] = data[i]$ artinya index output tergantung nilai-1 yang ada di count dengan index digit nya, kemudian nilai yang ada pada $count[digit]--$ dikurangi satu. Loop keempat adalah loop standar yang asign nilai output satu per satu ke data.

Analisis operasi

Array Access: Random sebanyak 927, Sorted sebanyak 927, Reverse Sorted sebanyak 927, Nearly Sorted sebanyak 927, Many Duplicates sebanyak 618, array acces dilakukan setiap kali asign value ke vector bantuan count dan juga saat memasukan value sorted ke vector bantuan ouput, begitu juga dengan data. Dengan demikian array acces menjadi metode operasi utama dari algoritma ini

Pengaruh bentuk data terhadap jumlah operasi.

Bentuk data nggak terlalu ngaruh. Jumlah pass ditentukan oleh digit max, bukan urutan data. Data yang udah sorted, acak, atau reverse tetep jalan 3 pass kalau max-nya 802. Yang beda mungkin distribusi digit tertentu, tapi Big-O tetap $O(N \times D)$ $D = \text{digit max element}$.

Analisis performa

Algoritma	Waktu Eks dengan N = 100 (Random)	Peringkat
Bubble Sort	0.023644 ms	6
Selection Sort	0.006803 ms	5
Insertion Sort	0.002064 ms	1
Merge Sort	0.008476 ms	4
Quick Sort	0.003567 ms	2
Radix Sort	0.006292 ms	3

Table 2. Table waktu eksekusi

"Berdasarkan tabel di atas, algoritma yang paling cepat untuk dataset random dengan $N=100$ adalah Insertion Sort, dengan waktu 0.002064 ms. Ini terjadi karena insertion sort memiliki mekanisme exit early dimana jika data yang mau di insert sudah pada posisi yang benar dia akan berhenti melakukan shift. Di sisi lain, Bubble Sort paling lambat dengan waktu 0.023644 ms. Ini karena algoritma ini memang memiliki notasi $O(n^2)$ dimana dia akan melakukan iterasi walaupun satu per satu untuk membandingkan data.

Ukuran Data	Algoritma tercepat	Alasan
Kecil $N = 100$	Insertion Sort 0.002064 ms	Karena insertion hanya perlu melakukan sedikit shift jika ukuran data nya kecil, dan dia juga punya mekanisme early exit pada loop shift nya.
Besar $N = 10000$	Radix Sort 0.482582 ms	Karena algoritma ini memiliki kompleksitas $O(n)$ dan karena hanya memiliki 1 jenis operasi yang simple.

Table 3. Table algoritma tercepat

Untuk data kecil insertion menjadi yang tercepat karena dia tidak perlu melakukan loop shift yang banyak hanya untuk mencari posisi satu data. Karena loop shift ini bekerja dengan shift satu per satu dari belakang hingga posisi yang benar ini membuat data yang dibelakang waktu loop nya jadi lama. Hal ini membuat cara loop dan shift ini menjadi kelemahan jika datanya banyak. Sedangkan Radix memiliki karakteristik yang stabil dimana waktu nya hanya dipengaruhi oleh n , dan tidak ada kesenjangan operasi antara n besar dan n kecil k . karena radix melakukan operasi array acces dan tidak melakukan comparasi seperti algoritma lain.

Bentuk Data	Algoritma	Alasan
Sangat Sensitif	Insertion Sort dan Quick Sort	Dua algoritma ini berlawanan dimana insertion semakin urut datanya semakin cepat, bebanding dengan quick sort jika data nya urut ata urut terbalik dia menjadi lambat
Tidak Sensitif	Merge Sort	Karena algoritma ini menggunakan divide membuat nya secara natural tidak terpengaruh pada

		bentuk data
--	--	-------------

Table 4. Table algoritma sensitif bentuk data

Tipe Data N = 10.000	Insertion Sort	Quick Sort	Merge Sort
Sorted	0.007464 ms	40.883075 ms	0.432948 ms
Reverse Sorted	87.947053 ms	46.149733 ms	0.576839 ms
Random	19.505480 ms	0.487881 ms	0.927542 ms
Nearly Sorted	5.566374 ms	0.694992 ms	0.768351 ms
Many Duplicates	44.108827 ms	0.463816 ms	1.043040 ms

Table 5. Table algoritma sensitif bentuk data waktu

Karena insertion ini melakukan loop shift yang memiliki karakteristik semakin kekanan data nya semakin banyak shift yang perlu dilakukan, karakteristik ini membuat algoritma ini sangat tidak efisien untuk tipe data reverse sorted yang besar, sebaliknya jika data nya sorted algoritma ini sangat cepat. Yang menarik disini adalah quick sort dimana semakin data nya sorted semakin tidak efisien. Sedangkan merge sorted sangat stabil tidak peduli tipe data ataupun besar data nya. Hal ini karena dia melakukan divide and conquer, dimana data akan terus di divide menjadi 2 sampai data nya sisa 1 dan kemudian dia baru melakukan merge.

Algoritma	Big-O	N=100	N=1.000	N=10.000	Sesuai teori?
Bubble Sort	$O(n^2)$	0.023644	1.715169	216.477113	Ya dengan lonjakan 72 dan 9100
Selection Sort	$O(n^2)$	0.006803	0.287483	25.555336	Lebih Sedikit dengan lonjakan 42 dan 3700
Insertion Sort	$O(n^2)$	0.002064	0.133402	19.505480	Ya dengan lonjakan 64 dan 9400
Merge Sort	$O(n \log n)$	0.008476	0.078769	0.927542	Lebih Sedikit dengan lonjakan 9 dan 109

Quick Sort	$O(n \log n)$	0.003567	0.044243	0.487881	Ya dengan lonjakan 12 dan 136
Radix Sort	$O(n)$	0.006292	0.041498	0.482582	Lebih Sedikit dengan lonjakan 6 dan 76

Table 6. Table hubungan teori dan hasil

Validasi: Apakah waktu eksekusi naik sesuai nilai Big-O?

Untuk $O(n^2)$: kalau N 10 kali lipat ($100 \rightarrow 1.000 \rightarrow 10.000$), waktunya seharusnya naik sekitar 100 kali lipat (10^2).

Untuk $O(n \log n)$: kalau N 10 kali lipat, waktunya naik sekitar $10 \times (\log 10N / \log N)$ kali, kurang lebih 13-15 kali.

Untuk $O(n)$: waktunya naik linear (10 kali lipat kalau N 10 kali lipat).

Untuk beberapa algoritma, seperti bubble, insertion, dan quick hasil big-o cukup akurat karena rasio kenaikan waktu pada skala besar ($N = 10.000$) sudah mendekati prediksi teori, di mana Bubble dan Insertion menunjukkan lonjakan sekitar 9.000–9.500 kali (mendekati teori $N^2 = 10.000$ kali) serta Quick Sort mengikuti pola $N \log N$; sedangkan Selection Sort memiliki rasio yang lebih rendah karena jumlah swap-nya selalu tetap sebesar $N-1$ sehingga beban operasi pertukaran mendominasi sebagian waktu eksekusi di semua skenario, Merge Sort hasilnya lebih lambat dalam rasio karena setiap pemanggilan merge harus membuat dan menyalin subvector tambahan yang membebani eksekusi di setiap level rekursi, dan Radix Sort hasilnya berbeda dari big-o $O(N)$ karena jumlah pass-nya bergantung pada jumlah digit nilai maksimum (D), sehingga kompleksitas yang relevan adalah $O(N \times D)$ yang membuat rasio pertumbuhannya berbeda sedikit dari big-o.

TAUTAN GIT

Repo lengkap : <https://github.com/biahilil/Algoritma-dan-Struktur-Data.git>
Linked List : <https://github.com/DSA26-ULM/module-4-sorting-biahilil.git>